

Image Recognition using Convolutional Neural Networks on the CIFAR-10 Dataset

A Project Report

Submitted by: **Archi Jain**

Date: 6th September 2026

1. Abstract

This project implements an image Recognition System using a Convolutional Neural Network (CNN) trained on the CIFAR-10 dataset, a benchmark consisting of 60,000 colour images of size 32×32 pixels, spread evenly across 10 object categories. The objective was to build a model capable of automatically recognising the correct category of an unseen image, and to understand how architectural choices like convolutional layers, pooling, batch normalisation, dropout, and data augmentation affect a model's ability to generalise to new data. The model was trained for 25 epochs using the Adam optimizer and real-time data augmentation, and evaluated on a held-out test set of 10,000 images. The final model achieved a test accuracy of 82.36% and a test loss of 0.529. Performance was further analysed using a per-class classification report and a confusion matrix, which revealed that the model performs strongly on structurally distinct classes such as automobile, ship, and frog (recall above 0.90), while classes with overlapping visual features particularly cat, bird, and dog were more frequently confused with one another.

2. Introduction

Image Recognition System is teaching a computer to correctly label what is shown in a photograph and is one of the foundational problems in computer vision. Unlike simpler benchmark datasets such as MNIST, which contains grayscale handwritten digits with high contrast and minimal background variation, real-world images involve colour, texture, lighting variation, and cluttered backgrounds, all of which make classification considerably more difficult.

The CIFAR-10 dataset was chosen for this project because it sits at a useful difficulty level: complex enough to require a genuine CNN architecture (rather than a simple fully-connected network) to achieve reasonable performance, yet small and standardised enough to train on a single machine within a practical amount of time. It is also one of the most widely used benchmark datasets in the deep learning community, providing a large body of prior work to compare results against.

Convolutional Neural Networks were selected as the modelling approach because, unlike a traditional fully-connected (Dense) network, they are specifically designed to exploit the spatial structure of image data, the fact that neighbouring pixels are related and together form meaningful patterns such as edges, textures, and shapes. This property makes CNNs both more accurate and more parameter-efficient than flat networks for image-based tasks.

3. Objective

1. Build a Convolutional Neural Network capable of classifying 32×32 colour images into one of 10 object categories.
2. Apply appropriate data preprocessing (pixel normalisation, one-hot label encoding) to prepare the raw dataset for training.
3. Use real-time data augmentation to improve the model's ability to generalise, given the relatively small size of the training set.
4. Evaluate the trained model using standard classification metrics, and analyse its errors using a confusion matrix.

5. Document the reasoning behind each architectural and preprocessing decision made during the project.

4. Dataset Description

The CIFAR-10 dataset consists of 60,000 colour images, each 32×32 pixels with 3 colour channels (Red, Green, Blue), divided into 50,000 training images and 10,000 test images. The images are evenly distributed across 10 mutually exclusive classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. Each class containing exactly 6,000 images (5,000 training and 1,000 test). Because the dataset is perfectly balanced across classes, overall accuracy is a fair and reliable evaluation metric for this project, unlike imbalanced classification problems where accuracy alone can be misleading.

The dataset was loaded directly through Keras's built-in dataset utilities (`tensorflow.keras.datasets.cifar10`), and a random sample of training images was inspected before any preprocessing was applied, shown in Figure 1.

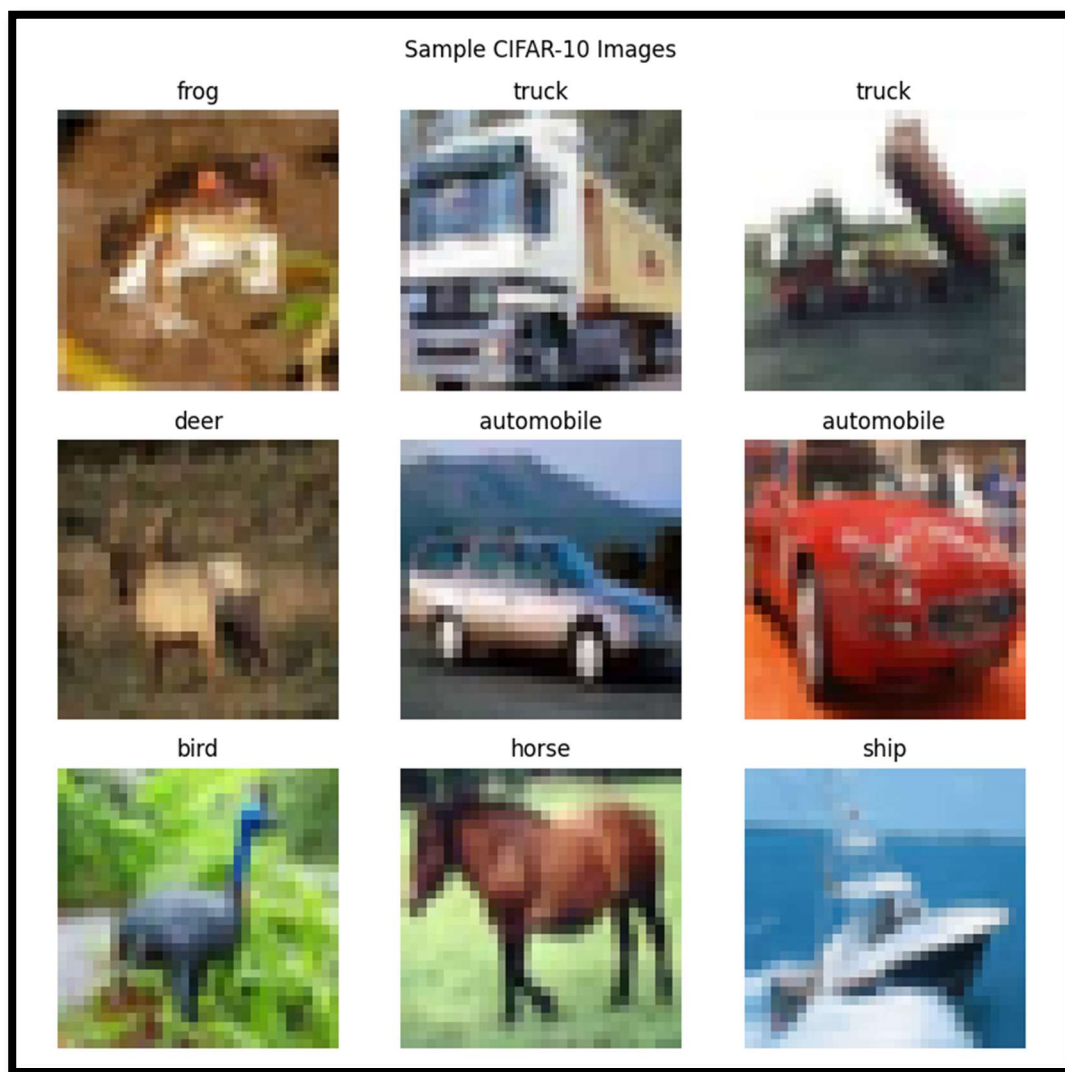


Figure 1. A random sample of nine CIFAR-10 training images with their true class labels.

5. Methodology

5.1 Data Preprocessing

Two preprocessing steps were applied before training.

5.1.1 Pixel Normalisation

Pixel Normalization is a process of scaling down the values to some extent. In the dataset, raw pixel values range from 0 to 255, so each pixel value was divided by 255.0, rescaling every value into the range [0, 1]. This is conceptually identical to Min/Max scaling, using the known fixed minimum (0) and maximum (255) of image pixel intensities. This step improves training stability, since neural networks train more efficiently when input values are on a small, consistent scale rather than large raw integers.

5.1.2 One-Hot Encoding of Labels

In the dataset, the model's output layer uses a softmax activation function, producing one probability per class so the target labels were converted from a single integer (0–9) into one-hot encoded vectors (e.g., class 3 becomes [0,0,0,1,0,0,0,0,0]), so that the loss function could directly compare the predicted probability distribution against the true label.

5.2 Model Architecture

The model follows the standard CNN pattern of alternating feature extraction (convolution and pooling) with classification (fully connected Dense layers). Three convolutional blocks were used, with the number of filters increasing with depth (32 → 64 → 128). Each block consisted of two Conv2D layers with Batch Normalisation (which rescales the outputs of a layer to keep training stable, especially helpful in deeper networks.) applied after each, followed by MaxPooling2D to progressively reduce spatial resolution (32×32 → 16×16 → 8×8 → 4×4), and a Dropout layer to reduce overfitting by randomly deactivating a fraction of neurons during each training step.

Increasing the filter count with depth reflects the intuition that early layers detect simple, generic patterns (edges, colour transitions), while deeper layers combine these into more complex, class-specific patterns, which requires a larger number of filters to represent. After the final convolutional block, the resulting feature maps were flattened into a single vector and passed through a Dense layer (256 neurons, ReLU activation) followed by a final softmax output layer with 10 neurons. The complete architecture, with 668,842 total trainable parameters, is summarised in Table 1.

Table 1. Model architecture summary.

Layer (type)	Output Shape	Param #
Conv2D (32 filters)	(32, 32, 32)	896
BatchNormalization	(32, 32, 32)	128
Conv2D (32 filters)	(32, 32, 32)	9,248
BatchNormalization	(32, 32, 32)	128
MaxPooling2D	(16, 16, 32)	0
Dropout (0.25)	(16, 16, 32)	0
Conv2D (64 filters)	(16, 16, 64)	18,496

Layer (type)	Output Shape	Param #
BatchNormalization	(16, 16, 64)	256
Conv2D (64 filters)	(16, 16, 64)	36,928
BatchNormalization	(16, 16, 64)	256
MaxPooling2D	(8, 8, 64)	0
Dropout (0.25)	(8, 8, 64)	0
Conv2D (128 filters)	(8, 8, 128)	73,856
BatchNormalization	(8, 8, 128)	512
MaxPooling2D	(4, 4, 128)	0
Dropout (0.3)	(4, 4, 128)	0
Flatten	(2048)	0
Dense (256)	(256)	524,544
BatchNormalization	(256)	1,024
Dropout (0.5)	(256)	0
Dense (10, softmax)	(10)	2,570

5.3 Data Augmentation

Because the CIFAR-10 training set is relatively small for the complexity of the task, and several classes are visually similar (e.g., cats and dogs, cars and trucks), the model was trained using real-time data augmentation via Keras's ImageDataGenerator. During training, each image was randomly rotated (up to 15 degrees), shifted horizontally and vertically (up to 10% of image size), and flipped horizontally. This exposes the model to a wider variety of realistic image variations without requiring additional labelled data, helping it learn features that generalise rather than memorising exact pixel arrangements from the training set.

5.4 Training Configuration

The model was compiled with the Adam optimizer and categorical crossentropy loss, and trained for 25 epochs with a batch size of 64. The full training configuration is summarised in Table 2.

Table 2. Training configuration and hyperparameters.

Hyperparameter	Value
Optimizer	Adam
Loss function	Categorical Crossentropy
Batch size	64
Epochs	25
Total trainable parameters	668,842
Validation data	Held-out CIFAR-10 test set (10,000 images), evaluated after each epoch
Data augmentation	Rotation ($\pm 15^\circ$), width/height shift (10%), horizontal flip

6. Results

6.1 Training and Validation Curves

Figure 2 shows the model's accuracy and loss over the 25 training epochs, tracked on both the training set and the held-out validation (test) set. Training accuracy increased steadily and smoothly from approximately 0.61 at epoch 1 to 0.81 by epoch 25, while training loss decreased correspondingly from approximately 1.11 to 0.56. Validation accuracy followed a similar overall upward trend, reaching a comparable final value of approximately 0.82, though it fluctuated noticeably from epoch to epoch (for example, dipping to around 0.66 at epoch 4 before recovering). This epoch-to-epoch noise in the validation curve, without a widening long-term gap between training and validation performance, suggests the model was not substantially overfitting. The data augmentation and dropout layers appear to have been effective in keeping the two curves broadly aligned by the end of training.

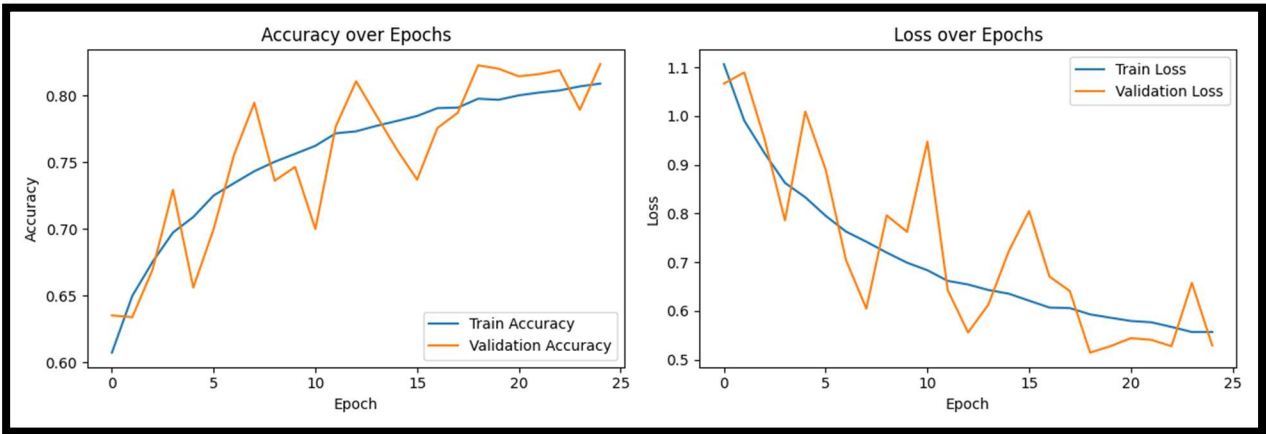


Figure 2. Training and validation accuracy (left) and loss (right) over 25 epochs.

6.2 Test Set Performance

The final model was evaluated on the complete 10,000-image test set, achieving a test accuracy of 82.36% and a test loss of 0.529. Table 3 presents the full per-class classification report, including precision, recall, and F1-score for each of the 10 categories.

Table 3. Per-class classification report on the test set.

Class	Precision	Recall	F1-score	Support
airplane	0.86	0.85	0.86	1000
automobile	0.89	0.95	0.92	1000
bird	0.87	0.67	0.76	1000
cat	0.80	0.59	0.68	1000
deer	0.81	0.79	0.80	1000
dog	0.85	0.71	0.77	1000
frog	0.62	0.97	0.76	1000
horse	0.88	0.87	0.88	1000
ship	0.90	0.92	0.91	1000
truck	0.88	0.92	0.90	1000
accuracy			0.82	10000
macro avg	0.84	0.82	0.82	10000

Class	Precision	Recall	F1-score	Support
weighted avg	0.84	0.82	0.82	10000

The strongest-performing classes were automobile (F1-score 0.92), ship (0.91), and horse (0.88) all classes with relatively distinctive shapes and consistent backgrounds. The weakest-performing class was cat (F1-score 0.68, recall only 0.59), indicating the model correctly identified fewer than 6 in 10 actual cat images. The frog class shows an unusual pattern: very high recall (0.97) but comparatively low precision (0.62), meaning the model rarely misses an actual frog, but frequently mislabels other classes particularly bird and cat as frog.

6.3 Confusion Matrix Analysis

Figure 3 shows the confusion matrix for all 10 classes on the test set, with rows representing the true class and columns representing the model's predicted class. The strongest diagonal values (correct predictions) appear for automobile (953), frog (969), ship (921), and truck (917), confirming these as the model's most reliably recognised categories.

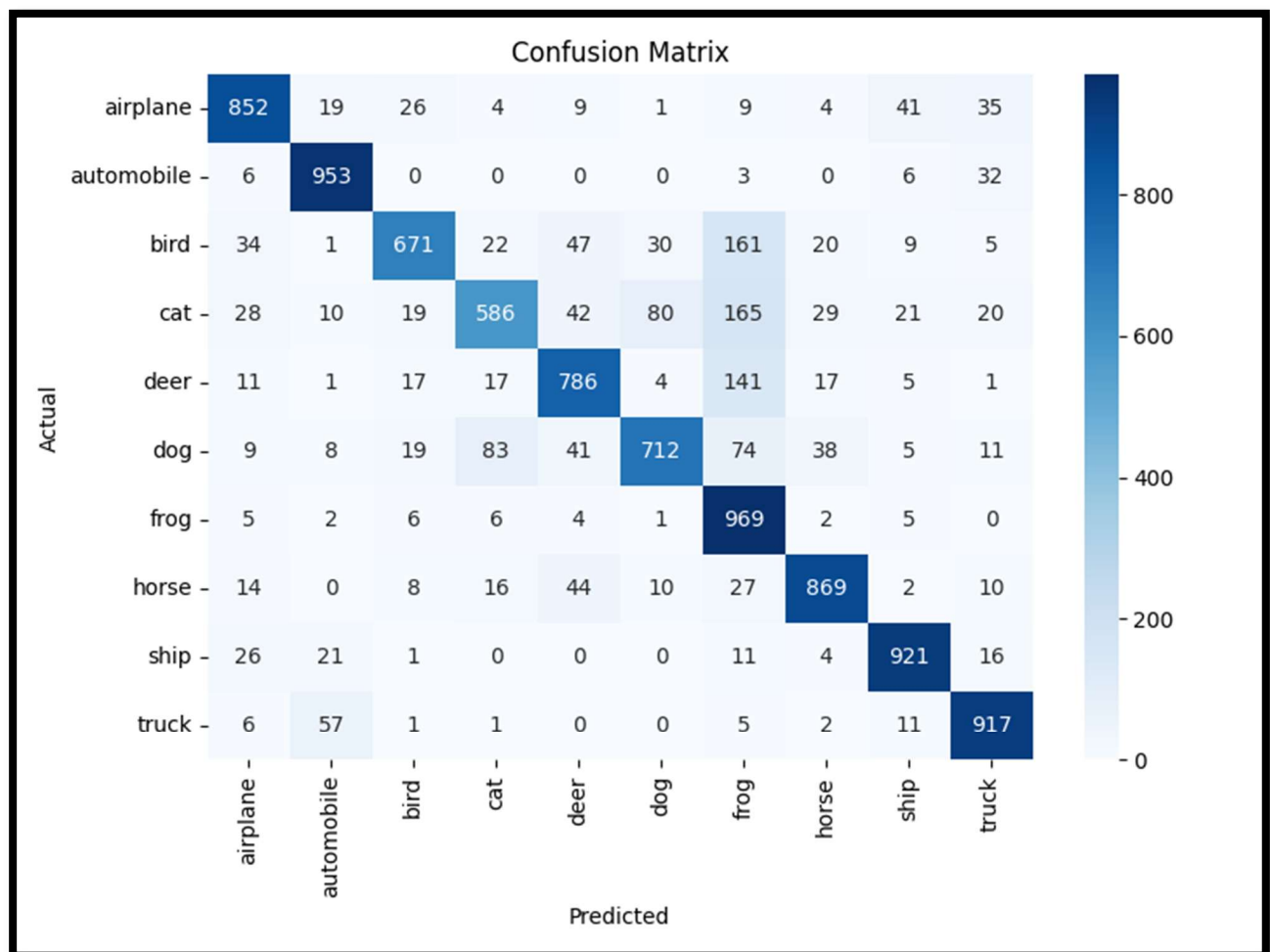


Figure 3. Confusion matrix showing predicted vs. actual class on the 10,000-image test set.

The most significant confusions were: 165 cat images misclassified as frog, 161 bird images misclassified as frog, 83 dog images misclassified as cat, 74 dog images misclassified as frog, and 57 truck images misclassified as automobile. Together, these patterns indicate that the model's errors are concentrated among animal classes that share overlapping visual features (body shape, texture, colour palette against natural backgrounds) rather than being distributed randomly across unrelated categories. This is a reassuring sign that the model is learning genuine visual features rather than exploiting spurious patterns in the data, since its confusions align with categories that are also visually similar to a human observer.

As a qualitative check, Figure 4 shows one individual test prediction: a cat image correctly classified as "cat" with 94.09% confidence, demonstrating the model's prediction pipeline functioning correctly end-to-end, from a saved model file through to a labelled prediction on unseen data.

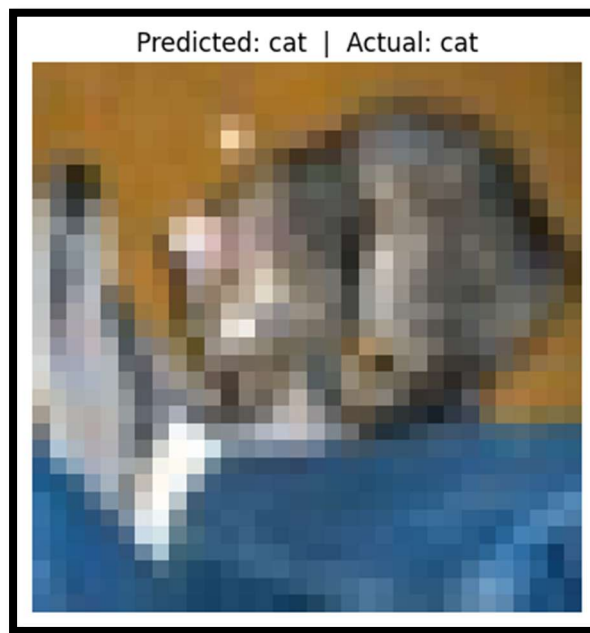


Figure 4. Example single-image prediction, correctly classified with high confidence.

7. Discussion

The results of this project illustrate several important principles of deep learning for image classification. First, the use of a CNN rather than a simple Dense network was essential: convolutional layers were able to detect local patterns such as edges and textures regardless of their position in the image, exploiting the spatial structure that a fully-connected network would otherwise discard.

Second, data augmentation and dropout appear to have meaningfully reduced overfitting. Although the validation accuracy curve was noisier than the training curve on an epoch-by-epoch basis, the two curves converged to similar final values (0.81 training vs. 0.82 validation), rather than training accuracy pulling far ahead of validation accuracy as would be expected in a clearly overfit model.

Third, the confusion matrix and per-class metrics reveal that not all classes are equally difficult. Classes with distinctive, consistent shapes and backgrounds (automobile, ship, truck) were classified reliably, while animal classes with overlapping visual characteristics (cat, bird, dog, frog) were more frequently

confused with one another. This pattern is consistent with published CIFAR-10 results using similarly-sized CNNs, and suggests that further improvement would likely require either a deeper architecture, transfer learning from a larger pretrained model, or additional feature-specific augmentation strategies targeted at distinguishing these animal classes.

8. Challenges Faced

- Training time: with 782 batches per epoch and approximately 360–400 seconds per epoch on the available hardware, full training across 25 epochs took roughly two to two-and-a-half hours, requiring the training process to run in the background while other work continued.
- Balancing model complexity against overfitting: choosing the number of convolutional blocks, filter counts, and dropout rates required iterative adjustment, too little regularisation risked overfitting given the relatively small training set, while too much risked underfitting and slower convergence.
- Interpreting a noisy validation curve: validation accuracy fluctuated considerably between epochs (for example, dropping from 0.73 at epoch 4 to 0.66 before recovering), which initially made it difficult to distinguish genuine overfitting from normal batch-to-batch noise; tracking the trend across the full curve rather than any single epoch was necessary to reach a confident conclusion.
- Compatibility issues between library versions: when combining ImageDataGenerator with `model.fit()` in newer Keras releases, which required adjusting how `steps_per_epoch` was specified to avoid the training loop running out of data partway through an epoch.

9. Conclusion

This project successfully implemented a Convolutional Neural Network for image classification on the CIFAR-10 dataset, achieving a test accuracy of 82.36%. The project demonstrated the complete deep learning workflow: data loading and preprocessing, CNN architecture design, training with data augmentation, and rigorous evaluation using both aggregate accuracy metrics and detailed per-class confusion matrix analysis. Beyond the specific numerical results, the project reinforced several core principles of applied deep learning i.e the importance of matching model architecture to data structure (CNNs for spatially structured image data), the value of regularisation techniques (dropout, batch normalisation, data augmentation) in controlling overfitting on a limited training set, and the importance of examining per-class and confusion-matrix-level results rather than relying on a single overall accuracy figure to understand model behaviour.

THANK YOU